



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Pipeline II: Hazards

5-Stage RISC-V Pipeline

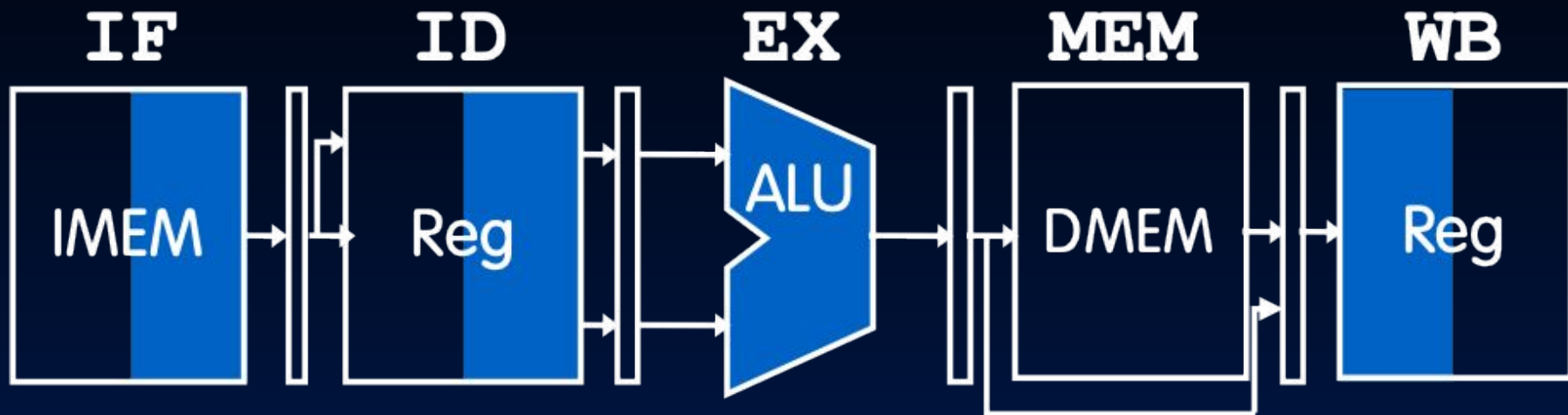
Instruction
Fetch (IF)

Instruction
Decode (ID)

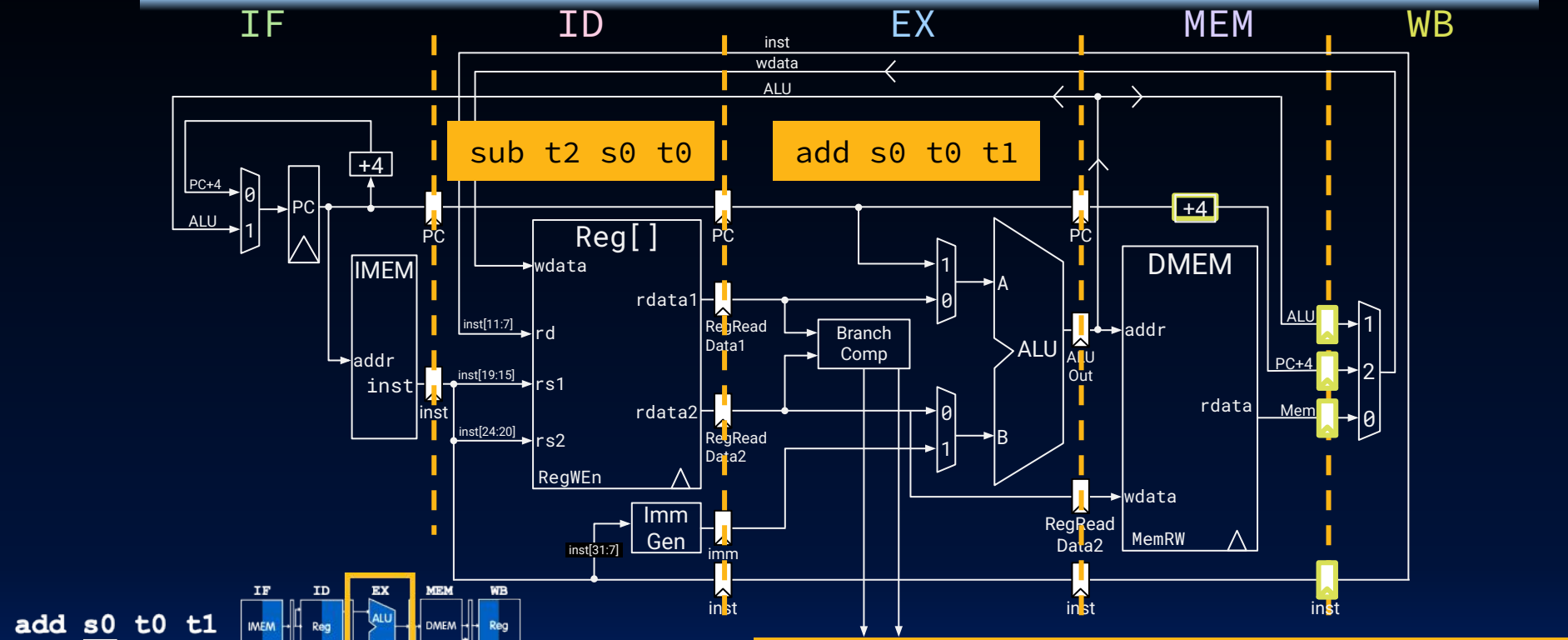
Execute (EX)

Memory
Access (MEM)

Write back
to Reg (WB)



Pipeline registers allow multiple instructions to execute in separate stages.

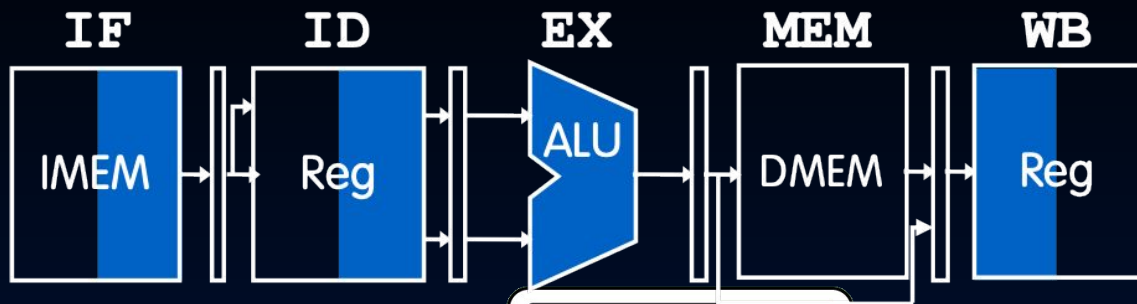


`add s0 t0 t1`

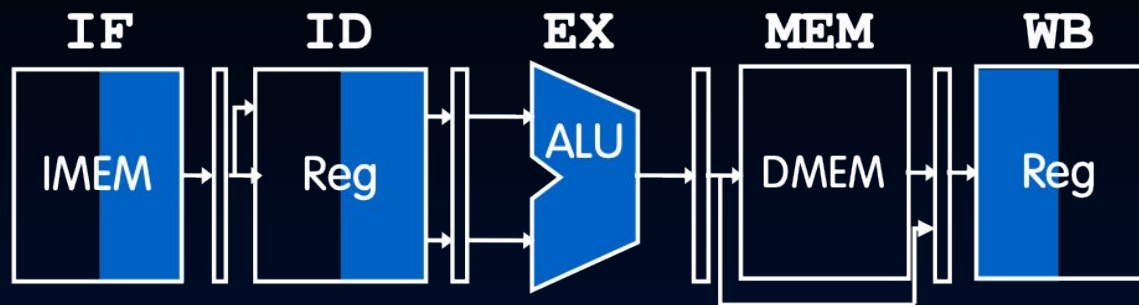
`sub t2 s0 t0`

Pipeline registers allow multiple instructions to execute in separate stages.

Pipeline Hazards Ahead!!!



Waterfall Diagram



time →

	1	2	3	4	5	6	7	8	9
add t0 t1 t2	IF	ID	EX	MEM	WB				
lw t0 8(t3)		IF	ID	EX	MEM	WB			
or t3 t4 t5			IF	ID	EX	MEM	WB		
sw t0 4(t3)				IF	ID	EX	MEM	WB	
sll t6 t0 t3					IF	ID	EX	MEM	WB

Can we read from memory twice in the same clock cycle?

How does sw's EX stage get the right value of t3 if or's WB stage hasn't executed yet??

How do branches work???



Three Types of Pipeline Hazards

A **hazard** is a situation in which a planned instruction cannot execute in the “proper” clock cycle.

1. Structural hazard

- a. Hardware does not support access across multiple instructions in the same cycle.

2. Data hazard

- a. Instructions have data dependency.
- b. Need to wait for previous instruction to complete its data read/write.

3. Control hazard

- a. Flow of execution depends on previous instruction.

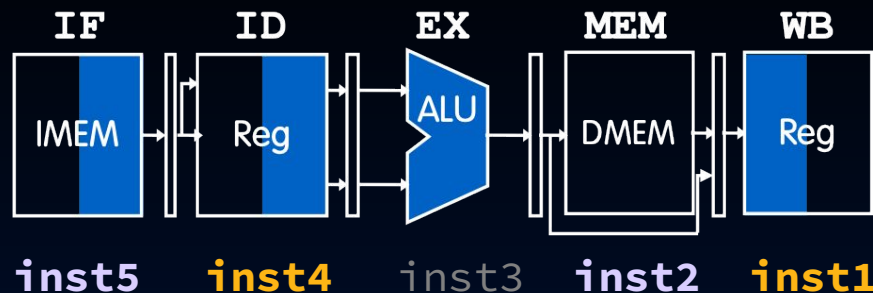


Structural Hazards

- Structural Hazards
- Data Hazards I
- Data Hazards II: Forwarding
- Data Hazards III: Load
- Control Hazards

Avoiding Structural Hazards

- Structural hazards are about the **hardware**, not the instruction.
- Two potential structural hazards in RV32I:
 - Simultaneous **RegFile** accesses: `inst4`, `inst1`
 - Simultaneous **memory** accesses: `inst5`, `inst2`.



In our current CPU, **structural hazards are not an issue**. We have the needed HW requirements on RegFile and Memory:

- RegFile supports simultaneous R/W
- IMEM, DMEM separate (caches)

Read at home. 🏠

SP26



Data Hazards

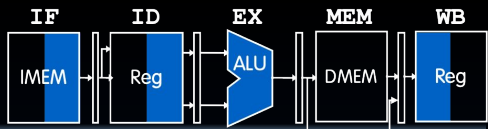
- Structural Hazards
- Data Hazards I
- Data Hazards II: Forwarding
- Data Hazards III: Load
- Control Hazards



Solutions to Hazards

1. Specifying **hardware requirements**.
2. **Stalling** is one inefficient solution to resolving any type of hazard.
 - a. Instructions get delayed, and performance suffers.
 - b. We'd like to avoid stalling where possible...!
3. **Forwarding**: Make extra connections in the datapath and instead use results when computed ("bypassing").
4. **Code scheduling**: Rearrange instructions at compile-time.

Revisit and review this slide.



Data hazard

- Instructions have data dependency.
- Need to wait for previous instruction to complete its data read/write.

	1	2	3	4	5	6	7	8	9
add s0 t0 t1	IF	ID	EX	MEM	<u>WB</u>				
sub t2 s0 t0		IF	<u>ID</u>	EX	MEM	WB			

Two programs shown.

	1	2	3	4	5	6	7	8	9
add t0 t1 t2	IF	ID	EX	MEM	<u>WB</u>				
lw t0 8(t3)		IF	ID	EX	MEM	WB			
or t3 t4 t5			IF	ID	EX	MEM	WB		
sw t0 4(t6)				IF	<u>ID</u>	EX	MEM	WB	
sll t6 t0 t3					IF	ID	EX	MEM	WB

Does the highlighting indicate a **data hazard**?





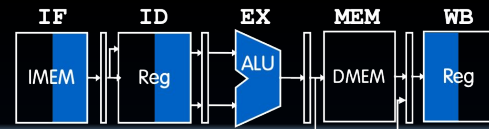
#1: Data Hazard

1. t=5: add writes s0.
t=3: sub reads s0. Stale value!

	1	2	3	4	5	6	7	8	9
add s0 t0 t1	IF	ID	EX	MEM	<u>WB</u>				
sub t2 s0 t0		IF	<u>ID</u>	EX	MEM	WB			



Stalling Instructions



- **Stalling**: Some instructions stall until resources are “ready.”
 - For #1, stall three cycles

	1	2	3	4	5	6	7	8	9
add s0 t0 t1	IF	ID	EX	MEM	<u>WB</u>				
sub t2 s0 t0		IF	<u>ID</u>	EX	MEM	WB			

	1	2	3	4	5	6	7	8	9
add s0 t0 t1	IF	ID	EX	MEM	<u>WB</u>				
nop		IF	–	–	–				
nop			–	–	–	–	–		
nop				–	–				
sub t2 s0 t0					IF	<u>ID</u>	EX	MEM	WB

Requires **extra pipeline state** to prevent unintended state changes in stalled stages (e.g., register/memory writes).



#2: Data Hazard...?

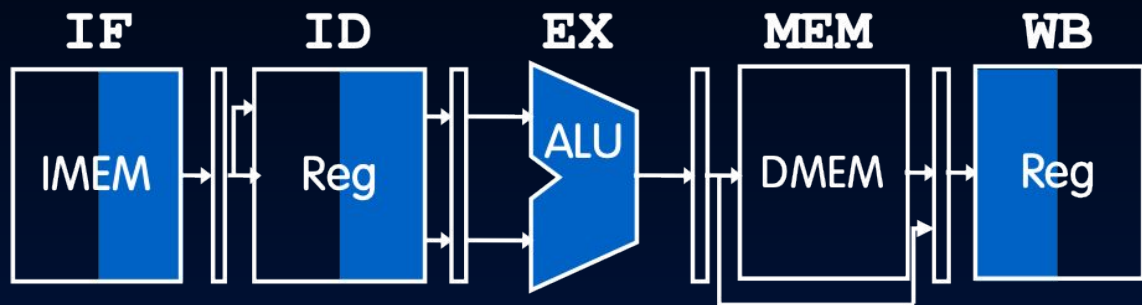
2. t=5: add writes t0.
t=5: sw reads t0. Stale value...???

If RegFile write occurs on NEXT cycle (t =6), then data hazard!
(this is our current implementation)

	1	2	3	4	5	6	7	8	9
add t0 t1 t2	IF	ID	EX	MEM	<u>WB</u>				
lw t7 8(t3)		IF	ID	EX	MEM	WB			
or t3 t4 t5			IF	ID	EX	MEM	WB		
sw t0 4(t6)				IF	<u>ID</u>	EX	MEM	WB	
sll t6 t0 t3					IF	ID	EX	MEM	WB

Increasing Requirements on RegFile HW

- Require RegFile HW to **write-then-read** in same cycle.
 - Achieved by writing on the falling edge (vs. rising)
 - Exploits high speed of RegFile (100 ps + 100 ps)



Indicated by shading in diagram.

- In some designs, e.g., high frequency, this RegFile design might be impossible.
 - Always check problem assumptions!!!

#2: If RegFile is write-then-read, then...

2. $t=5$, first half: add writes t_0
 $t=5$, second half: sw reads t_0 .

No data hazard if HW requirement is met!

	1	2	3	4	5	6	7	8	9
add $t_0 \ t_1 \ t_2$	IF	ID	EX	MEM	<u>WB</u>				
lw $t_7 \ 8(t_3)$		IF	ID	EX	MEM	WB			
or $t_3 \ t_4 \ t_5$			IF	ID	EX	MEM	WB		
sw $t_0 \ 4(t_6)$				IF	<u>ID</u>	EX	MEM	WB	
sll $t_6 \ t_0 \ t_3$					IF	ID	EX	MEM	WB

#1: If RegFile is write-then-read, then...

In situation #1, we can just stall **two** cycles to avoid data hazard.

1. t=5, first half: add writes s0.

t=5, second half: sub reads s0. Stale value!

	1	2	3	4	5	6	7	8	9
add s0 t0 t1	IF	ID	EX	MEM	<u>WB</u>				
sub t2 s0 t0		IF	<u>ID</u>	EX	MEM	WB			



	1	2	3	4	5	6	7	8	9
add s0 t0 t1	IF	ID	EX	MEM	<u>WB</u>				
nop		IF	–	–	–				
nop			–	–	–	–	–		
sub t2 s0 t0				IF	<u>ID</u>	EX	MEM	WB	



Data Hazards II: Forwarding

- Structural Hazards
- Data Hazards I
- Data Hazards II: Forwarding
- Data Hazards III: Load
- Control Hazards



Solutions to Hazards

1. Specifying **hardware requirements**.
2. **Stalling** is one inefficient solution to resolving any type of hazard.
 - a. Instructions get delayed, and performance suffers.
 - b. We'd like to avoid stalling where possible...!
3. **Forwarding**: Make extra connections in the datapath and instead use results when computed ("bypassing").
4. **Code scheduling**: Rearrange instructions at compile-time.



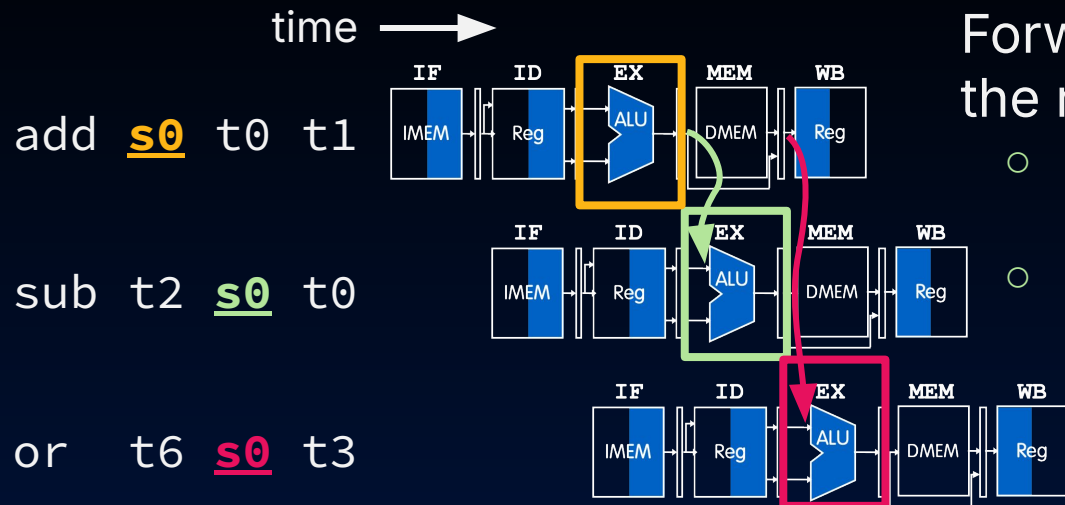
When do we actually need data?

Problem: Instruction depends on add's RegFile write in WB

	1	2	3	4	5	6	7	8	9
add <u>s0</u> t0 t1	IF	ID	<u>EX</u>	MEM	<u>WB</u>				
sub t2 <u>s0</u> t0		IF	ID	<u>EX</u>	MEM	WB			
or t6 <u>s0</u> t3			IF	ID	<u>EX</u>	MEM	WB		

- Stalling assumption earlier:
 - sub, or: read old value of s0 in ID, calculates wrong result in EX.
- But we really only need the right value during sub, or's EX (for ALU)...

Forwarding: Add Bypass Wires, More MUXes



Forwarding, aka bypassing, uses the result when it is computed.

- Don't wait for value to be stored into RegFile.
- Instead, grab operand from the pipeline stage.

Implementation:

- Make extra connections in the datapath.
- Also add forwarding control logic.

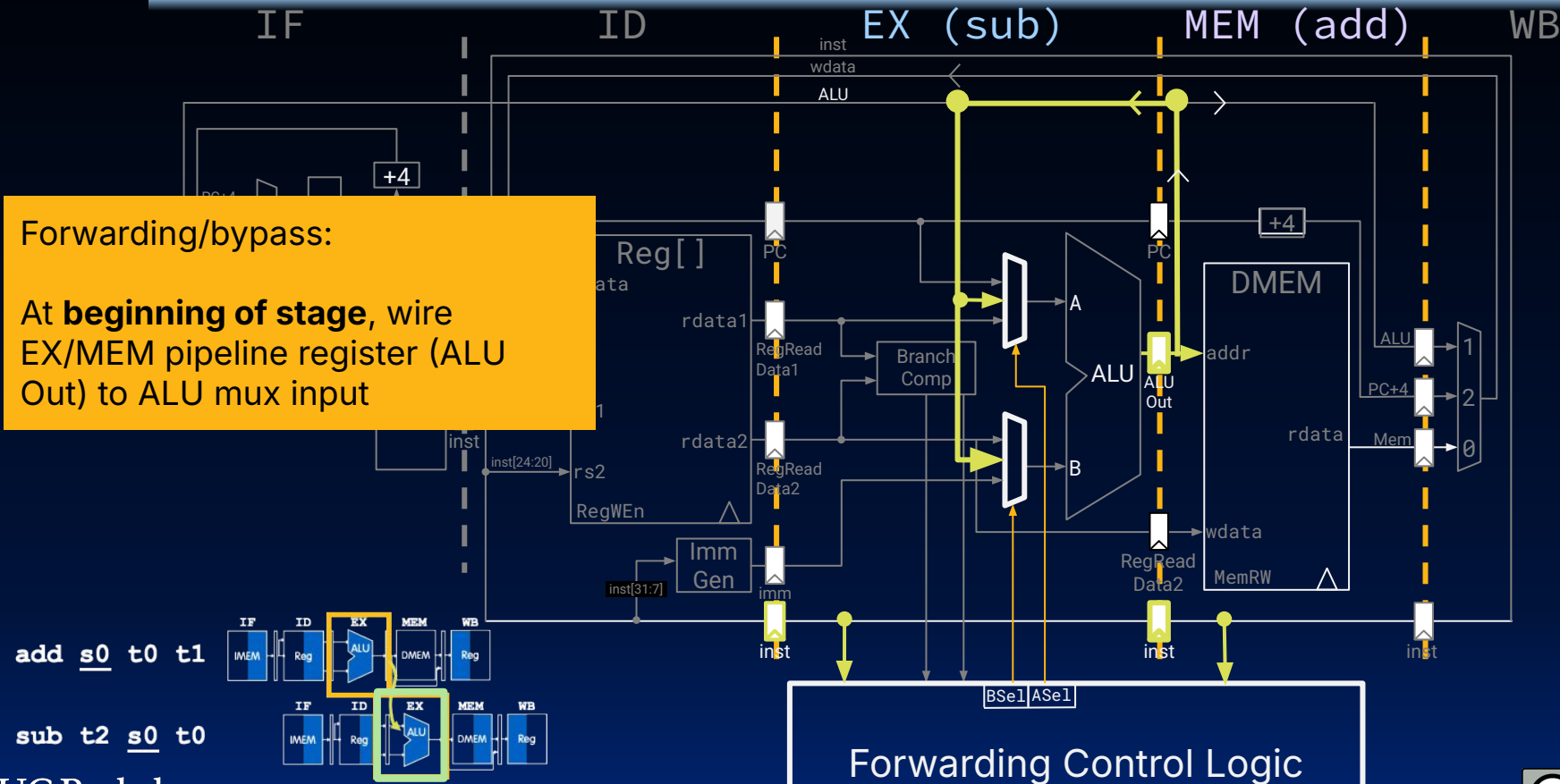
s0 value

5	5	5	5	5/9	9	9
---	---	---	---	-----	---	---

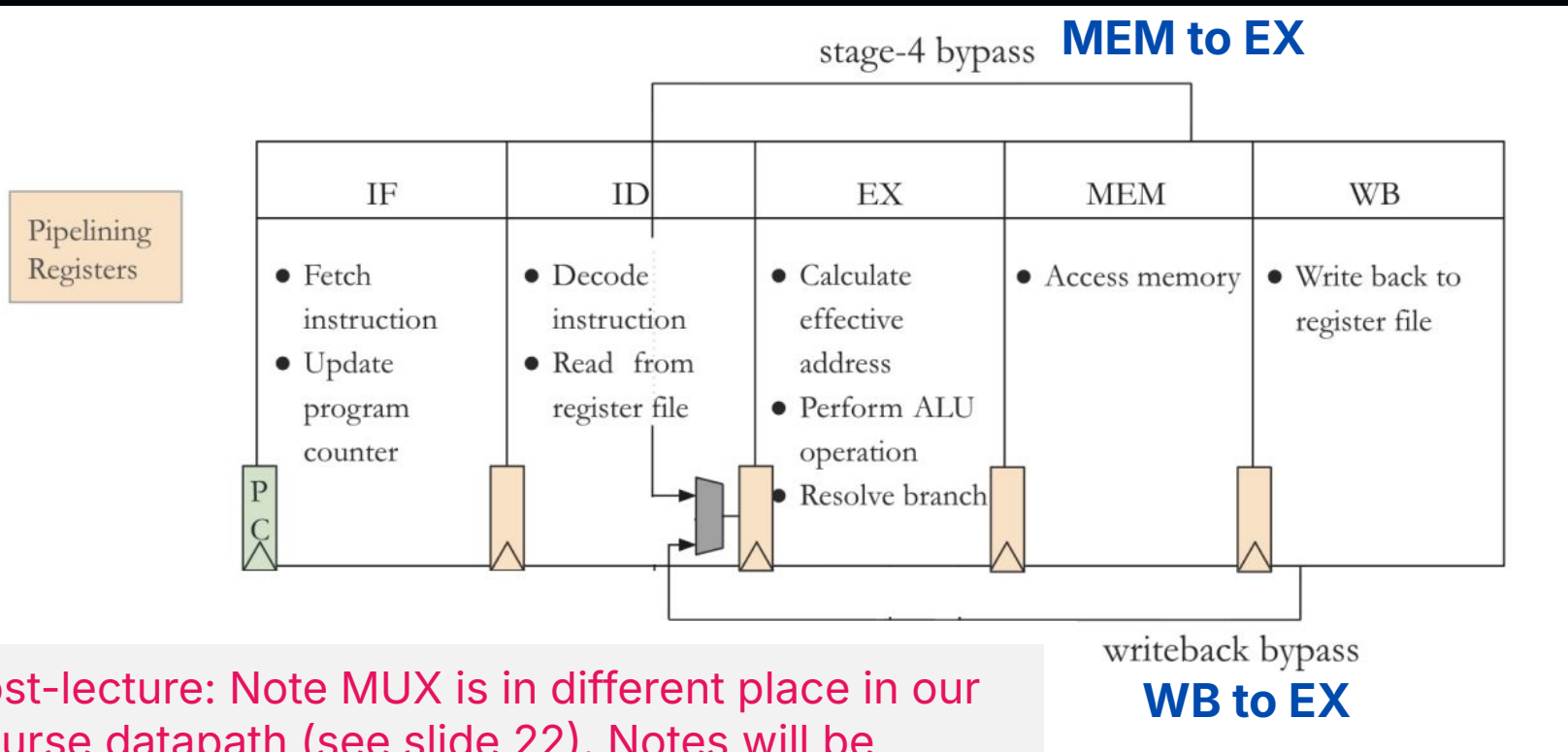
EX → EX..? No, actually MEM → EX

Forwarding/bypass:

At **beginning of stage**, wire EX/MEM pipeline register (ALU Out) to ALU mux input



Two Forwarding Paths in our Course



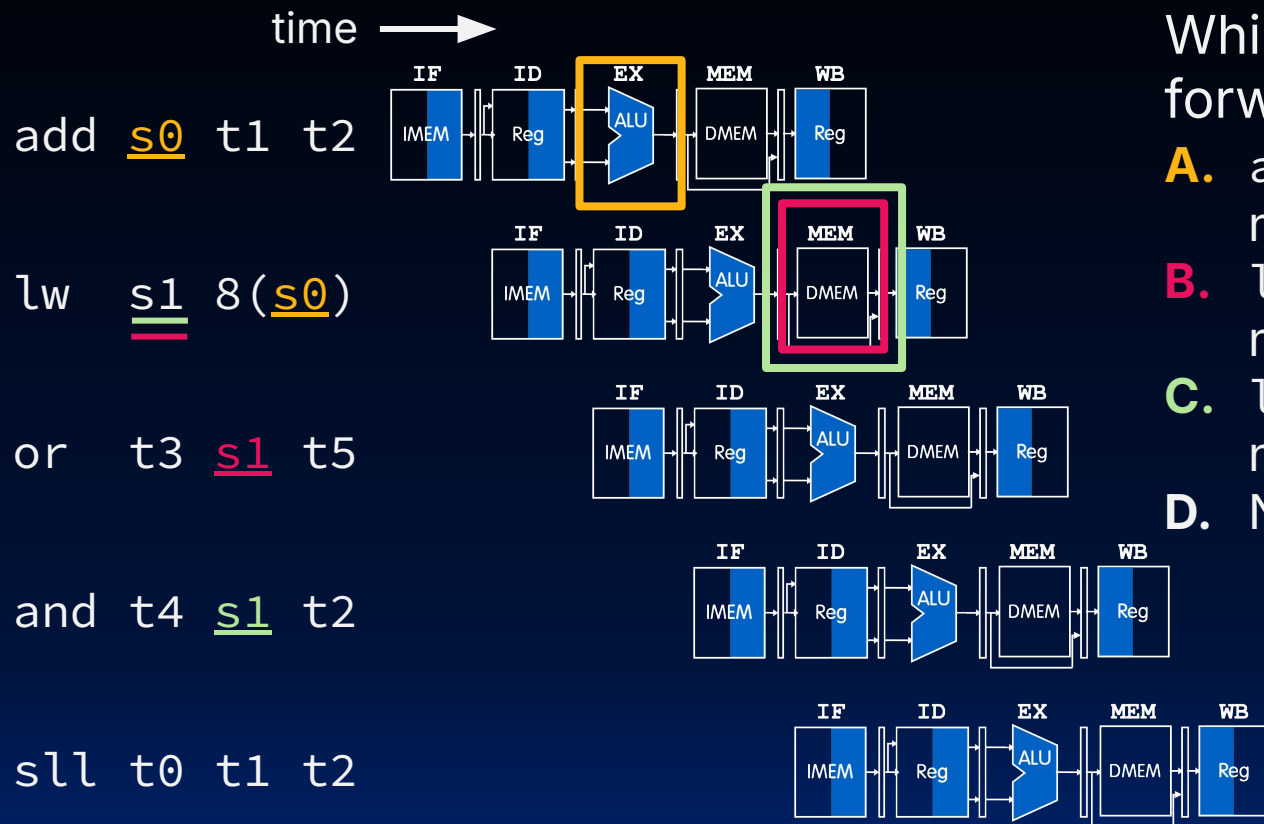
Post-lecture: Note MUX is in different place in our course datapath (see slide 22). Notes will be updated with the correct version of Slide 23 chart.



Data Hazards III: Load

- Structural Hazards
- Data Hazards I
- Data Hazards II: Forwarding
- Data Hazards III: Load
- Control Hazards

(Potential) Pipelined Hazards in RV32I Datapath



Which data hazards can forwarding fix? Select all:

- A.** add EX stage output needed by lw MEM input
- B.** lw MEM stage output needed by or EX input
- C.** lw MEM stage output needed by and EX input
- D.** None of the above

Which data hazards can forwarding fix?

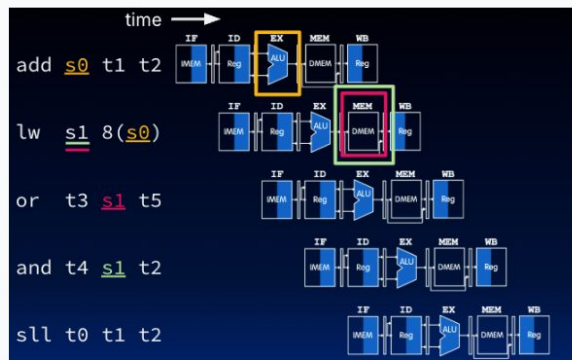
	1	2	3	4	5	6	7	8	9
add <u>s0</u> t1 t2	IF	ID	EX	MEM	WB				
lw <u>s1</u> 8(<u>s0</u>)		IF	ID	EX	MEM	WB			
or t3 <u>s1</u> t5			IF	ID	EX	MEM	WB		
and t4 <u>s1</u> t2				IF	ID	EX	MEM	WB	
sll t0 t1 t2					IF	ID	EX	MEM	WB

Select all:

- A. add EX stage output needed by lw MEM input
- B. lw MEM stage output needed by or EX input
- C. lw MEM stage output needed by and EX input
- D. None of the above



L28 Which Data Hazards Can Forwarding Fix?



A. add EX stage output needed by lw MEM input

0%

B. lw MEM stage output needed by or EX input

0%

C. lw MEM stage output needed by and EX input

0%

D. None of the above

0%



Which data hazards can forwarding fix?

	1	2	3	4	5	6	7	8	9
add <u>s0</u> t1 t2	IF	ID	EX	MEM	WB				
lw <u>s1</u> 8(<u>s0</u>)		IF	ID	EX	MEM	WB			
or t3 s1 t5			IF	ID	EX	MEM	WB		
and t4 <u>s1</u> t2				IF	ID	EX	MEM	WB	
sll t0 t1 t2					IF	ID	EX	MEM	WB

✓ **A.** (MEM→EX) Forward EX stage output to input of EX stage on next clock cycle.

✓ **C.** (WB → EX) Forward MEM stage output to input of EX stage on next clock cycle.

Which data hazards can forwarding fix?

	1	2	3	4	5	6	7	8	9
add s0 t1 t2	IF	ID	EX	MEM	WB				
lw <u>s1</u> 8(s0)		IF	ID	EX	<u>MEM</u>	WB			
or t3 <u>s1</u> t5			IF	ID	<u>EX</u>	MEM	WB		
and t4 s1 t2				IF	ID	EX	MEM	WB	
sll t0 t1 t2					IF	ID	EX	MEM	WB

✗ **B.** MEM stage (lw)'s output needed as EX stage (or)'s input in the **same clock cycle**.

Loads can result in an unavoidable pipeline stall.



Data Hazard: Loads

- The instruction slot after a load is called the **load delay slot**.
- If this instruction uses the result of load:
 - The hardware must **stall for one cycle** (plus **forwarding**).
 - This results in performance loss!

	1	2	3	4	5	6	7	8	9
add <u>s0</u> t1 t2	IF	ID	EX	MEM	WB				
lw <u>s1</u> 8(s0)		IF	ID	EX	MEM	WB			
or → nop			IF	ID	—	—	—		
or t3 <u>s1</u> t5				IF	ID	EX	MEM	WB	
and t4 s1 t2					IF	ID	EX	MEM	WB

Forwarding sends data in the **next clock cycle**.
Cannot go backwards in time!



Solution (1/2): Code Scheduling

C code

```
A[3] = A[0] + A[1];
```

```
A[4] = A[0] + A[2];
```

&a	a0
A[0]	t0
A[1]	t1
A[2]	t2
A[3]	t3
A[4]	t4



Simple compilation

(**13 cycles** for 7 instr.)

```
lw    t0    0(a0)
```

```
lw    t1    4(a0)  ↘ stall+forward: (+1 cycle)
```

```
add   t2    t0 t1
```

```
sw    t2    12(a0)
```

```
lw    t3    8(a0)  ↘ (+1 cycle)
```

```
add   t4    t0 t3
```

```
sw    t4    16(a0)
```

Solution (2/2): Code Scheduling

- Idea: Fix this hazard at the **code compilation stage**.
 - In the delay slot, put an instruction unrelated to the load result.
 - No performance loss!

Code scheduling:

With knowledge of the underlying CPU pipeline, the compiler reorders code to improve performance.

Simple compilation
(13 cycles for 7 instr.)

```
lw  t0  0(a0)
lw  t1  4(a0)
add  t2  t0 t1
sw   t2  12(a0)
lw  t3  8(a0)
add  t4  t0 t3
sw   t4  16(a0)
```

✓ Alternative
(11 cycles):

```
lw  t0  0(a0)
lw  t1  4(a0)
lw  t3  8(a0)
add  t2  t0 t1
sw   t2  12(a0)
add  t4  t0 t3
sw   t4  16(a0)
```

forward (+0 cycle)



[at home] Control Hazards

- Structural Hazards
- Data Hazards I
- Data Hazards II: Forwarding
- Data Hazards III: Load
- Control Hazards



Three Types of Pipeline Hazards

[review]

A **hazard** is a situation in which a planned instruction cannot execute in the “proper” clock cycle.



Structural hazard

- a. Hardware does not support access across multiple instructions in the same cycle.



Data hazard

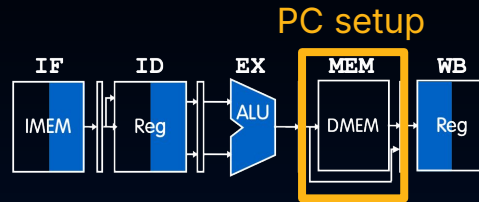
- a. Instructions have data dependency.
- b. Need to wait for previous instruction to complete its data read/write.

3. Control hazard

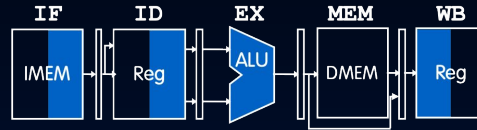
- a. Flow of execution depends on previous instruction.

Control Hazard: Conditional Branches

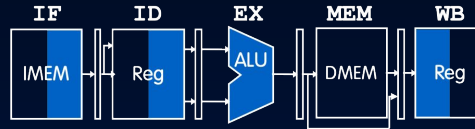
0x40
beq t0 t1 Label



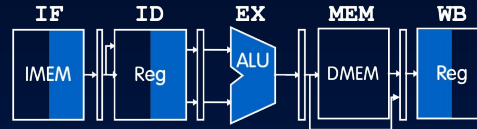
0x44
sub t2 s0 t0



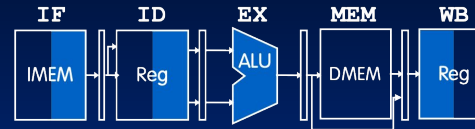
0x48
or t6 a0 t0



0x4c
xor t5 t1 s0



0x70
sw s0 8(t3) # Label



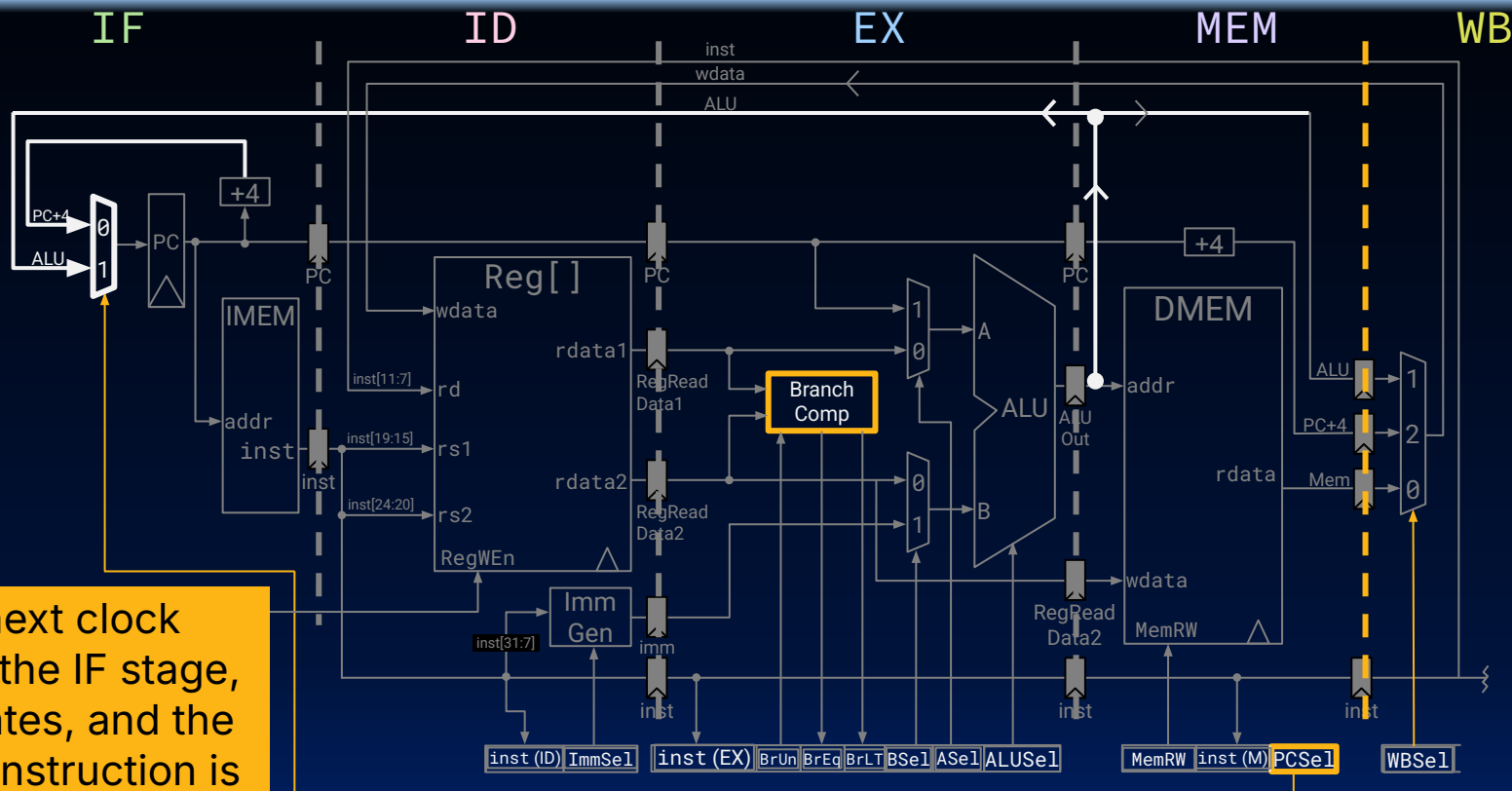
Control hazards occur when the instruction fetched may not be the one needed.

If the beq branch is **taken**, these three instructions have started executing before branch outcome is unknown.

correct instruction

Branch Results Ready During MEM

*clarification in slide notes

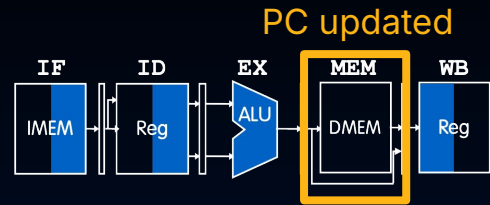


On the next clock cycle in the IF stage, PC updates, and the correct instruction is fetched.



If Branch Taken, Noop 3 instructions

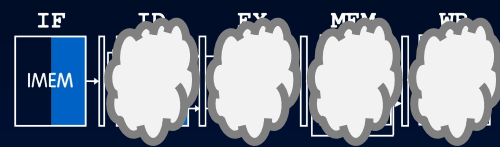
0x40
beq t0 t1 Label



0x44
sub t2 s0 t0



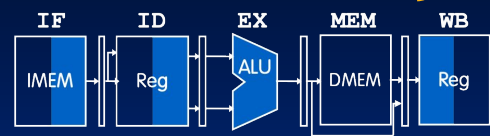
0x48
or t6 a0 t0



0x4c
xor t5 t1 s0



0x70
sw s0 8(t3) # Label



Flush pipeline by
converting 3
incorrect
instructions to
nops.

correct
instruction

Branch Prediction to Reduce Penalties

- Every taken branch in the simple RV32I pipeline costs **3 clock cycles**.
 - Branch results are computed during MEM.
 - (Note: if branch is not taken, then pipeline is not stalled; the correct instructions are correctly fetched sequentially after the branch instruction.)
 - (See textbook for an **out of scope** hardware cost improvement.)
- **Branch prediction** can improve the average CPU performance.
 - Early in the pipeline, guess which way branches will go.
 - Flush pipeline if branch prediction was incorrect.

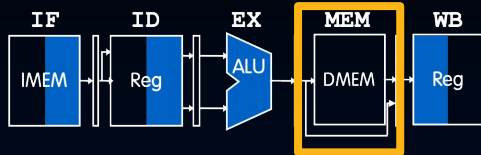
Naïve Predictor Interpretation: Don't Take Branch

"Guess" PC to be PC + 4

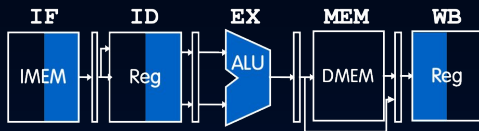
"Evaluate" Guess

The simple RV32I pipeline effectively always "predicts" that branches are **not taken**.

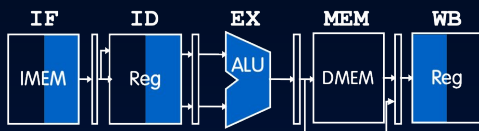
0x40
beq t0 t1 Label



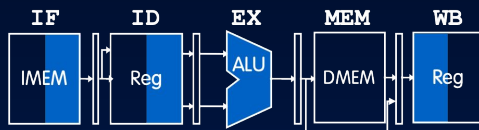
0x44
sub t2 s0 t0



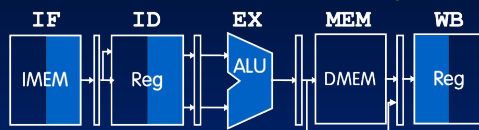
0x48
or t6 a0 t0



0x4c
xor t5 t1 s0



0x50
add t2 s0 s0

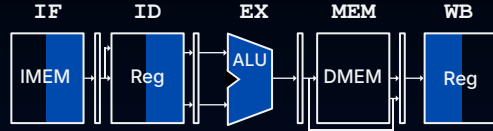


If branch not taken, correct instructions are already executed.

Penalty for incorrect prediction is still 3 clock cycles! Branch prediction tries to improve **average performance**.

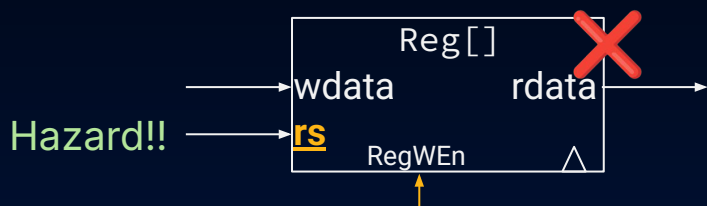


Diagram Reference

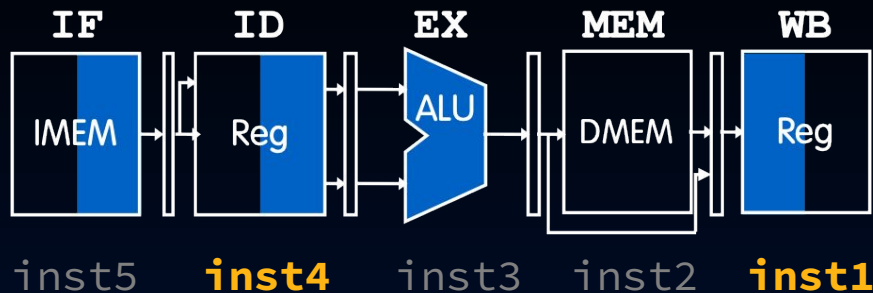
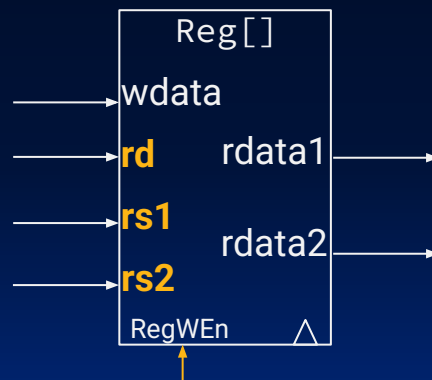


RV32I Avoids Structural Hazards (1/2): RegFile

- ✗ A structural hazard could occur if our RegFile hardware did not support simultaneous read/write:

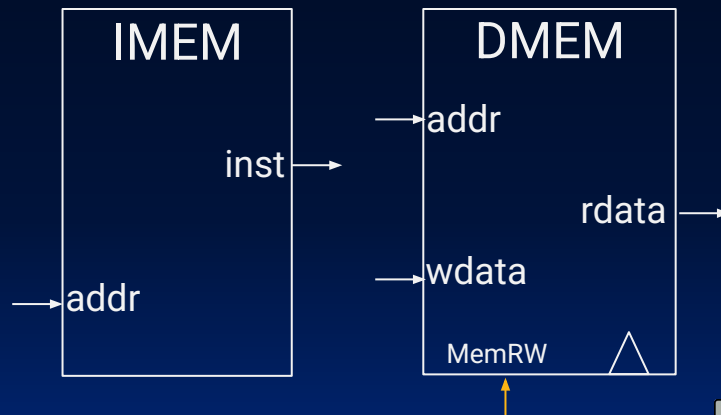
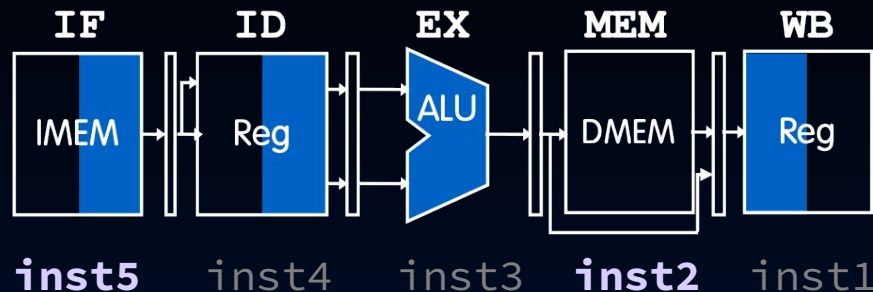


- RV32I's required RegFile design prevents this structural hazard.
 - Independent (separate) ports
 - Three accesses can happen in the same cycle** (2 read, 1 write).



RV32I Avoids Structural Hazards (2/2): Memory

- ✗ A structural hazard could occur if IMEM, DMEM were the same hardware:
 - Without separate memories, one of inst2 or inst5 would have to stall for a cycle.
- RV32I's required IMEM, DMEM design as separate memories prevents this structural hazard.
 - Remember: IMEM, DMEM are two L1 caches!





And in Conclusion...Pipelining and ISA Design

The RISC-V ISA is designed for **pipelining**:

- All instructions are 32 bits wide.
 - Easy to fetch (IF) and decode (ID), each in one clock cycle.
 - Contrast with CISC (Complex) x86: 1- to 15-byte-wide instructions!
- A small set of standard instruction formats
 - Can decode/read registers in one stage (2nd, ID).
- Load/store addressing conceptually:
 - Calculate address in 3rd stage (EX) with ALU; and
- Access memory in one stage (4th, MEM).
 - Memory operands are all aligned
 - Memory access takes only one cycle.

The **5-stage pipeline** we have studied is commonplace in many devices: cars, appliances, etc.